Part 1

Part I is designed for constructing variables.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib
from matplotlib import pyplot as plt
import operator
import statsmodels.api as sm
from tqdm import tqdm


SP500 = pd.read_csv('SP500_2022.csv')
Benchmark= pd.read_csv('Benchmark2022.csv')
Data = pd.read_csv('Data2022.csv')

Data['date'] = pd.to_datetime(Data['date'].astype(str), format='%Y%m%d')
Data['date'] = Data['date'].apply(lambda x:x.strftime('%Y%m'))
Data['date'] = Data['date'].astype(int)

Date_Unique = sorted(Data['date'].unique())    # create date index
Gvkey_Unique = sorted(Data['gvkey'].unique())  # create stock identifier index
idx_t_start = Date_Unique.index(197001)        # starting date of the backtesting
idx_t_end = Date_Unique.index(202212)          # ending date of the backtesting
Date_Unique = Date_Unique[idx_t_start: idx_t_end+1]

list_firm_date = []    # create a first list for each month
for idx_date in tqdm(Date_Unique):
    list_firm_date.append(Data[Data['date'].isin([idx_date])].sort_values(
        'gvkey',inplace=False).reset_index(drop=True))
```

```
100%|██████████████████████████████████████████████████████████████████████████| 6
36/636 [00:07<00:00, 88.49it/s]
```

In [2]:
```
Data.head()
```

Out[2]:

	gvkey	iid	tic	cusip	prccm	prchm	prclm	trt1m	fyearq	fqtr	ceqq	cshoq	sp500	date
0	1010	1	4165A	00099V004	73.0000	74.0000	66.1299	NaN	NaN	NaN	NaN	NaN	1	196201
1	1010	1	4165A	00099V004	73.2500	75.1299	70.5000	1.1986	NaN	NaN	NaN	NaN	1	196202
2	1010	1	4165A	00099V004	68.0000	75.0000	66.5000	-7.1672	NaN	NaN	NaN	NaN	1	196203
3	1010	1	4165A	00099V004	71.8799	73.5000	67.2500	5.7058	NaN	NaN	NaN	NaN	1	196204
4	1010	1	4165A	00099V004	59.6300	72.0000	53.2500	-16.1727	NaN	NaN	NaN	NaN	1	196205

Functions

Construct variables

In [3]:
```
# Makes sure we use only point-in-time data, no look-ahead bias
Factor_SP500 = []     # Whether the firm is a member of SP500
```

```python
Factor_HL1M = []      # Price Momentum
Factor_MV = []        # Market Cap
Factor_BM = []        # Book to Price
Factor_AdjPrice = [] # Adjusted Closing Price Prccm
Factor_RET = []       # Monthly Return, this is also the dependent variable

for idx_t in tqdm(Date_Unique):
    df_firm = list_firm_date[Date_Unique.index(idx_t)].copy(deep=True)

    # Construct the signal for each factor

    # SP500 Indicator: member==1, nonmember==0;
    Factor_SP500.append(df_firm[['sp500', 'gvkey']])

    df_firm['MV'] = np.log(df_firm['cshoq']*1000000*df_firm['prccm'])
    Factor_MV.append(df_firm[['MV', 'gvkey']]) # Log Market Cap
    df_firm = df_firm.drop(['MV'], axis=1)

    df_firm['BM'] = df_firm['ceqq']*1000000/(df_firm['cshoq']*1000000*df_firm['prccm'])
    Factor_BM.append(df_firm[['BM', 'gvkey']]) # Book to Price
    df_firm = df_firm.drop(['BM'], axis=1)

    df_firm['HL1M'] = (df_firm['prchm']-df_firm['prccm'])/(df_firm['prccm']-
                                                           df_firm['prclm'])
    Factor_HL1M.append(df_firm[['HL1M', 'gvkey']]) # 1M Price high-1M Price Low
    df_firm = df_firm.drop(['HL1M'], axis=1)

    Factor_AdjPrice.append(df_firm[['prccm', 'gvkey']])  #  Adjusted Closing Price

    Factor_RET.append(df_firm[['trt1m', 'gvkey']])  #  Monthly Return
```

```
 20%|                                                                                |
126/636 [00:01<00:08, 62.97it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 25%|                                                                                |
162/636 [00:02<00:07, 67.03it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 27%|                                                                                |
169/636 [00:02<00:06, 67.40it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 29%|                                                                              | 1
83/636 [00:02<00:07, 62.76it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\ar
raylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 30%|                                                                                |
190/636 [00:02<00:06, 64.53it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
```

```
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 31%|██████████████████████████                                                    |
197/636 [00:03<00:06, 63.61it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 32%|██████████████████████████                                                    |
204/636 [00:03<00:06, 63.12it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 34%|████████████████████████████                                                  |
218/636 [00:03<00:06, 66.10it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 35%|████████████████████████████                                                  |
225/636 [00:03<00:06, 66.14it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 36%|████████████████████████████████                                              |
232/636 [00:03<00:06, 66.94it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\a
rraylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
D:\ProgramData\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning:
divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
 38%|████████████████████████████████                                              | 2
39/636 [00:03<00:06, 65.72it/s]D:\ProgramData\anaconda3\lib\site-packages\pandas\core\ar
raylike.py:397: RuntimeWarning: divide by zero encountered in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
100%|████████████████████████████████████████████████████████████████████████████| 6
36/636 [00:09<00:00, 66.03it/s]
```

Part II： Sorting

In [4]:

```python
def portfolio_sort(Factor, Factor_RET, Factor_SP500, Quantile):
    RET_Sort_top = []
    RET_Sort_bottom = []
    Factor_Sort = []
    Qspread = []

    # Sort portfolio
    for idx_t in tqdm(range(len(Date_Unique)-1)):
        f = Factor[idx_t]     # the signal is formed at time t
        col_name = f.columns[0]

        # we look at the portfolio performance at time t+1
```

```
        ret_sp500 = Factor_RET[idx_t + 1]

        sp500 = Factor_SP500[idx_t]['sp500'].replace(0, np.nan)

        # we only keep firms in the S&P500
        f_sp500 = f[sp500.isnull().values==False].copy(deep=True)

        tmp =  pd.merge(f_sp500, ret_sp500, on='gvkey', how='outer')
        tmp.dropna(axis='index', how='any', inplace=True)
        tmp = tmp.sort_values(by=[col_name], ascending=False)
        f_sp500 = tmp[[col_name, 'gvkey']].reset_index(drop=True).copy(deep=True)
        ret_sp500 = tmp[['trt1m', 'gvkey']].reset_index(drop=True).copy(deep=True)

        if idx_t==0: # no first observation since the portfolio starts at time t+1
            RET_Sort_top.append(np.nan)
            RET_Sort_bottom.append(np.nan)
            Factor_Sort.append(np.nan)

        # portfolios are sorted into quantiles
        idx_quantile = np.ceil(len(ret_sp500)/Quantile)

        top = ret_sp500.loc[0:idx_quantile-1].copy(deep=True)
        bottom = ret_sp500.loc[(Quantile-1)*idx_quantile:].copy(deep=True)
        RET_Sort_top.append(top)
        RET_Sort_bottom.append(bottom)
        Factor_Sort.append(f_sp500)

    # Spread= Top-Bottom
    for idx in range(len(RET_Sort_top)):
        if idx == 0:
            Qspread.append(np.nan)
        elif len(RET_Sort_top[idx]) == 0:
            Qspread.append(np.nan)
        else:
            Qspread.append(np.nanmean(RET_Sort_top[idx]['trt1m'])
                           - np.nanmean(RET_Sort_bottom[idx]['trt1m']) )

    return RET_Sort_top, RET_Sort_bottom, Factor_Sort, Qspread
```

In [5]:

```
Quantile = 5;  # we can decide how many portfolios to separate.

# Compute the Factor QSpread for each factor
HL1M_RET_Sort_top,HL1M_RET_Sort_bottom,HL1M_Factor_Sort,HL1M_Qspread=portfolio_sort(
    Factor_HL1M,Factor_RET,Factor_SP500,Quantile)

MV_RET_Sort_top,MV_RET_Sort_bottom,MV_Factor_Sort,MV_Qspread=portfolio_sort(
    Factor_MV,Factor_RET,Factor_SP500,Quantile)

BM_RET_Sort_top,BM_RET_Sort_bottom,BM_Factor_Sort,BM_Qspread=portfolio_sort(
    Factor_BM,Factor_RET,Factor_SP500,Quantile)
```

```
100%|██████████████████████████████████████████████████████████████████████| 6
35/635 [00:06<00:00, 97.60it/s]
100%|██████████████████████████████████████████████████████████████████████| 6
35/635 [00:06<00:00, 96.37it/s]
100%|██████████████████████████████████████████████████████████████████████| 63
5/635 [00:06<00:00, 100.84it/s]
```

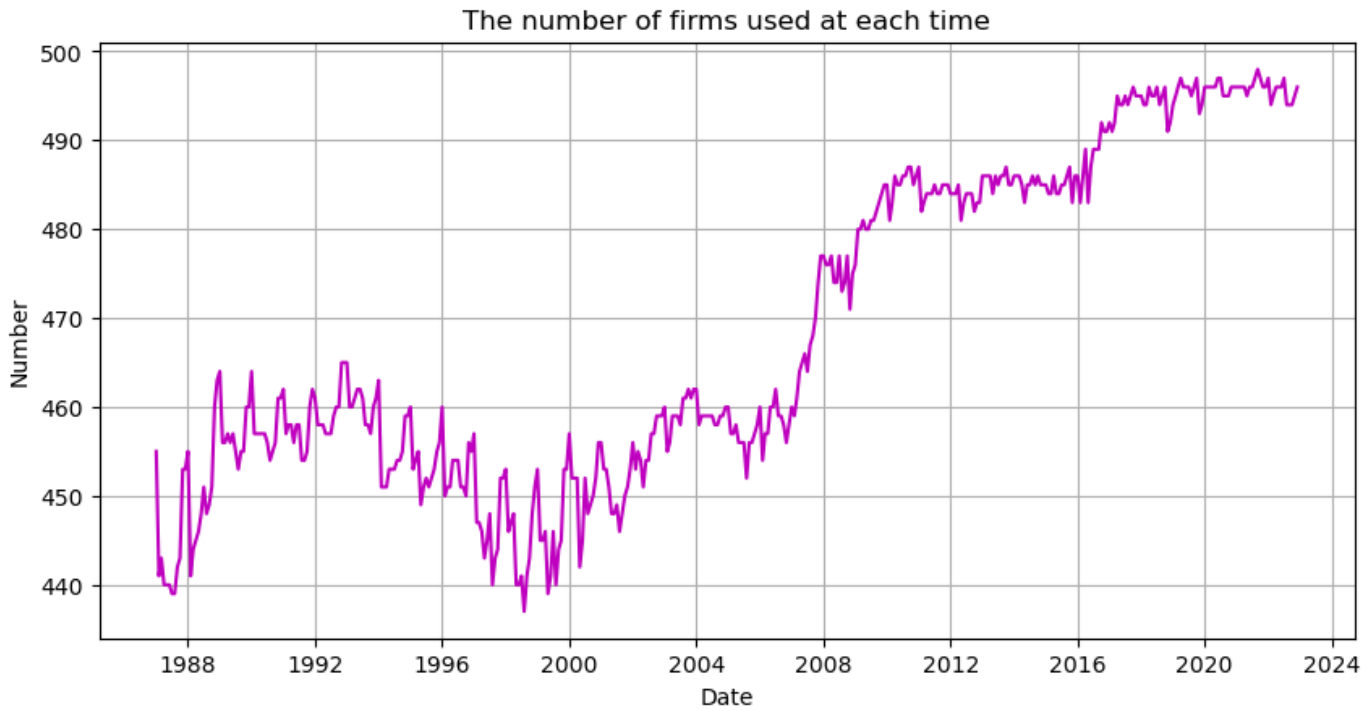
Q1:

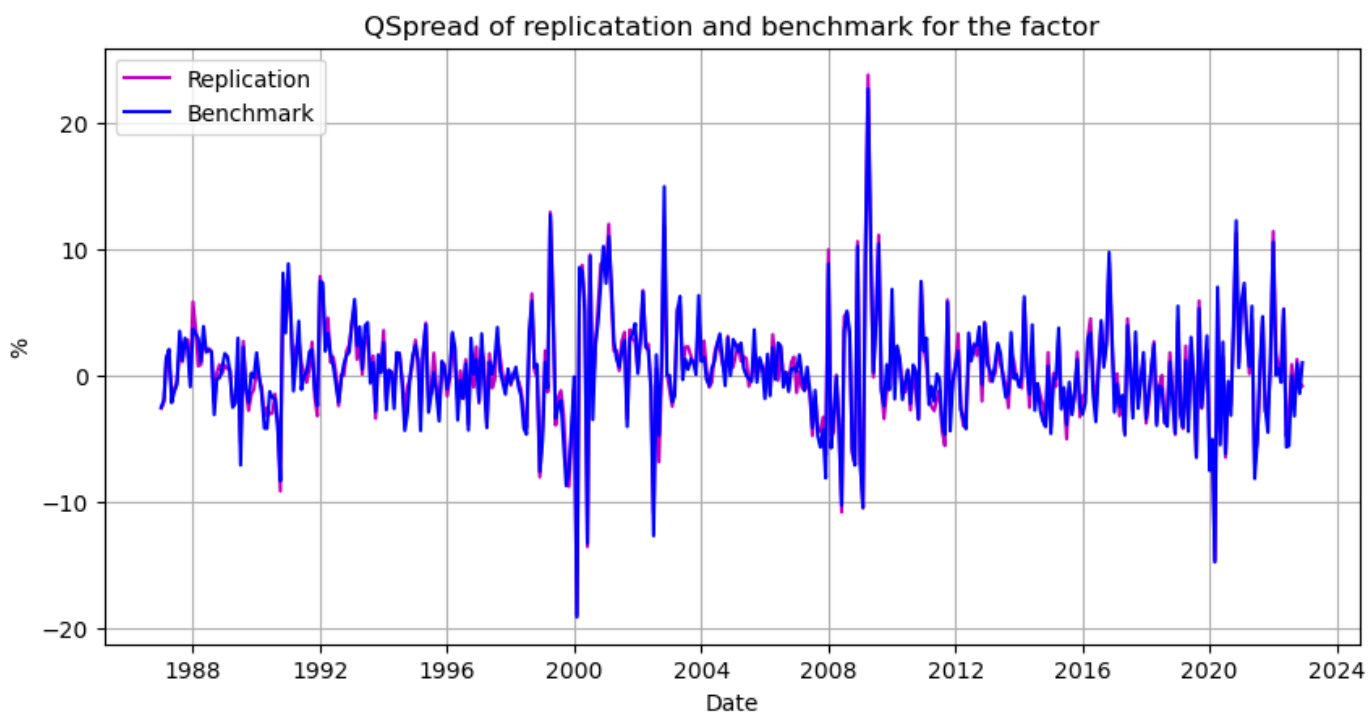
In [6]:

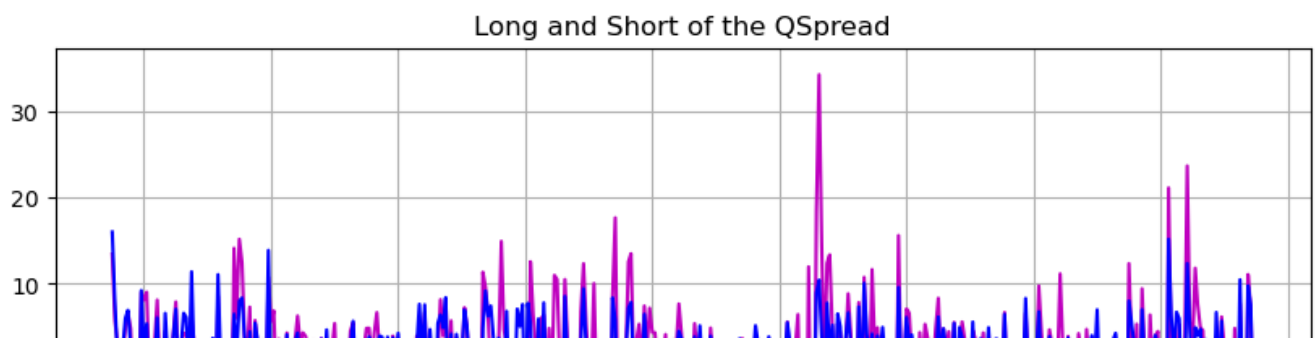
```
import datetime as dt
import matplotlib.dates as mdates
```

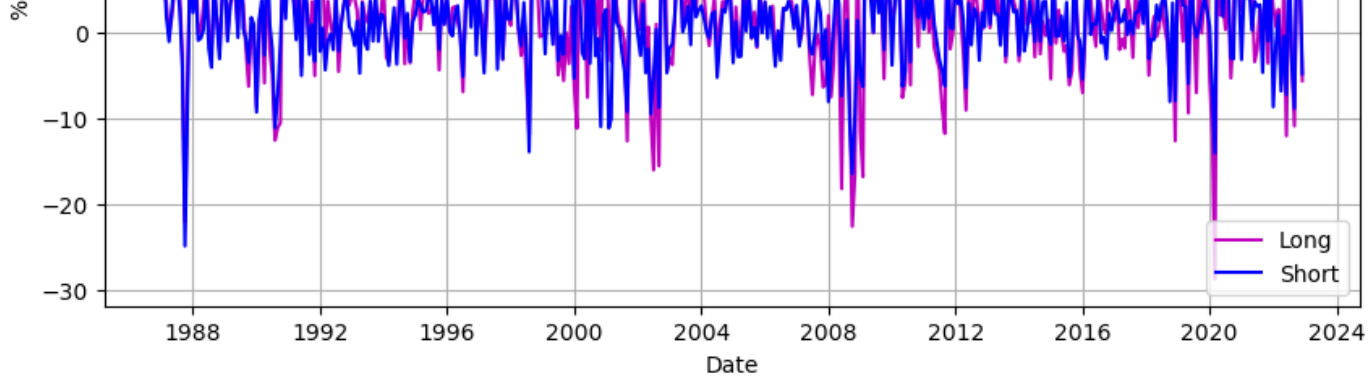
```
# compare whether we could replicate the factor returns from the exsiting Benchmark
benchmark = Benchmark[['Date', 'BP']].copy(deep=True)
benchmark['Date'] = pd.to_datetime(benchmark['Date'].astype(str), format='%Y%m%d')
benchmark['Date'] = benchmark['Date'].apply(lambda x:x.strftime('%Y%m'))

idx_t_start = Date_Unique.index(int(benchmark['Date'].iloc[0]))
idx_t_end = Date_Unique.index(int(benchmark['Date'].iloc[-1]))

test_factor = BM_Factor_Sort[idx_t_start:idx_t_end+1]
test_Qspread = BM_Qspread[idx_t_start:idx_t_end+1]
test_long = BM_RET_Sort_top[idx_t_start:idx_t_end+1]
test_short = BM_RET_Sort_bottom[idx_t_start:idx_t_end+1]

sp500 = SP500[idx_t_start:idx_t_end+1]

list_nfirms = []
list_long = []
list_short = []

for idx_t in range(len(test_factor)):
    list_nfirms.append(len(test_factor[idx_t]))

for idx_t in range(len(test_long)):
    list_long.append(np.nanmean(test_long[idx_t]['trt1m']))
    list_short.append(np.nanmean(test_short[idx_t]['trt1m']))

ret_sp500 = np.exp(np.cumsum(np.log(1+sp500['ret_sp500']))))-1
ret_Qspread = np.exp(np.cumsum(np.log(1+np.array(test_Qspread)/100)))-1
ret_Long  = np.exp(np.cumsum(np.log(1+np.array(list_long)/100)))-1
ret_Short = np.exp(np.cumsum(np.log(1+np.array(list_short)/100)))-1



plt.figure(figsize=(10,22))
plt.subplot(4,1,1)  # multiple plots in one figure
xvalues = [dt.datetime.strptime(d,"%Y%m").date() for d
           in np.asarray(benchmark['Date']).astype(str)]
plt.plot(xvalues, list_nfirms, '-m', label="The number of firms used at each time")
#plt.legend(loc='upper left')
plt.xlabel('Date')
plt.ylabel('Number')
plt.grid(True)
plt.title('The number of firms used at each time')

plt.subplot(4,1,2)
plt.plot(xvalues, test_Qspread, '-m', label="Replication")
plt.plot(xvalues, benchmark['BP'].values, '-b', label="Benchmark")
plt.legend(loc='upper left')
plt.xlabel('Date')
plt.ylabel('%')
plt.grid(True)
plt.title('QSpread of replicatation and benchmark for the factor')


plt.subplot(4,1,3)
plt.plot(xvalues, list_long, '-m', label="Long")
plt.plot(xvalues, list_short, '-b', label="Short")
plt.legend(loc='lower right')
plt.xlabel('Date')
plt.ylabel('%')
plt.grid(True)
plt.title('Long and Short of the QSpread')


plt.subplot(4,1,4)
plt.plot(xvalues, ret_sp500, '-m', label="SP500")
```

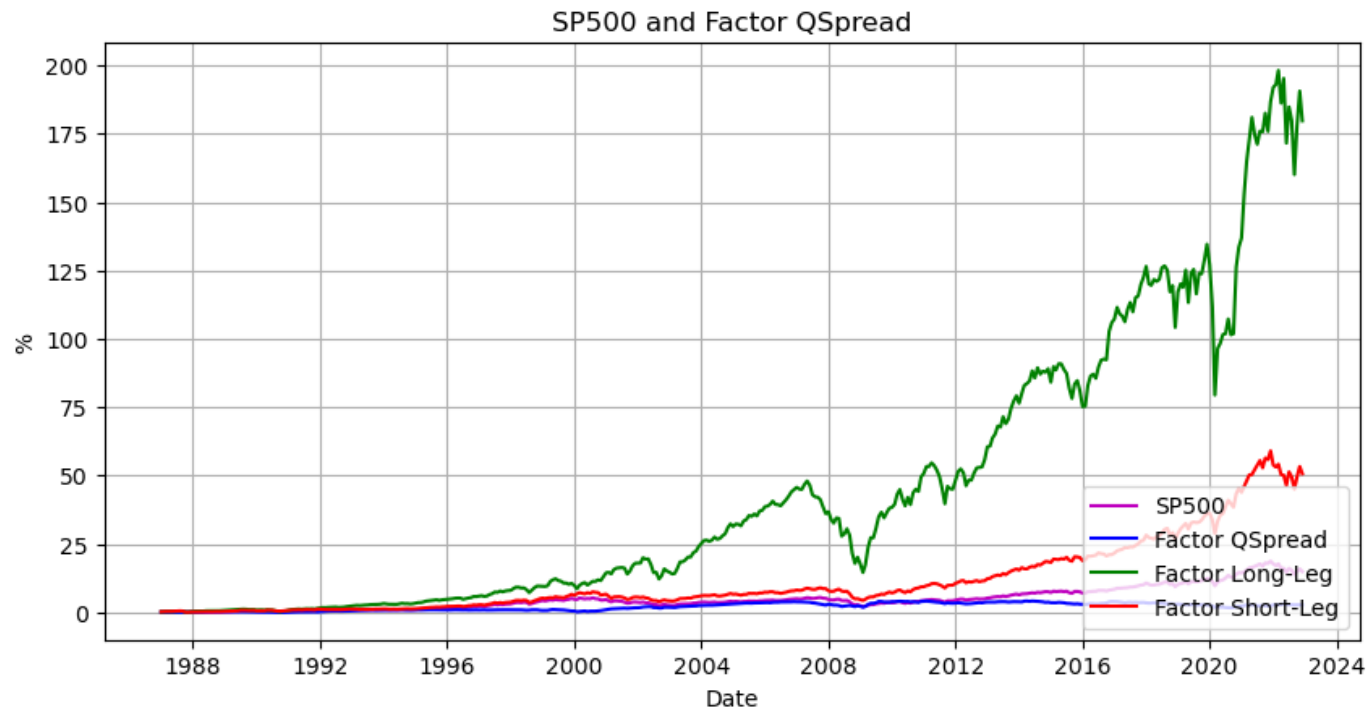
```
plt.plot(xvalues, ret_Qspread, '-b', label="Factor QSpread")
plt.plot(xvalues, ret_Long, '-g',label="Factor Long-Leg")
plt.plot(xvalues, ret_Short, '-r', label="Factor Short-Leg")
plt.legend(loc='lower right')
plt.xlabel('Date')
plt.ylabel('%')
plt.grid(True)
plt.title('SP500 and Factor QSpread')

plt.show()
```











Q2:

Compare the replication to the benchmark

Compute the correlation coefficients for each replicated factor with its benchmark;

Pick the same period of the benchmarket;

The factor to be tested: MV, BM, HL1M

The criteria is a correlation coefficient greater than 0.9 (if the correlation is negative, take the inverse)

In [7]:
```
from scipy import stats
list_factors = ['MV', 'BM', 'HL1M', 'S&P500']
test_MV_Qspread = MV_Qspread[idx_t_start:idx_t_end+1]
test_BM_Qspread = BM_Qspread[idx_t_start:idx_t_end+1]
test_HL1M_Qspread = HL1M_Qspread[idx_t_start:idx_t_end+1]


list_corr = []
list_corr.append(stats.pearsonr(test_MV_Qspread, Benchmark['LogMktCap'])[0])
list_corr.append(stats.pearsonr(test_BM_Qspread, Benchmark['BP'])[0])
list_corr.append(stats.pearsonr(test_HL1M_Qspread, Benchmark['HL1M'])[0])
```

```
df_corr = pd.DataFrame(list(zip(list_factors, list_corr)), columns =['Factor', 'Corr'])
df_corr
```

Out[7]:

	Factor	Corr
0	MV	0.992169
1	BM	0.973885
2	HL1M	0.971786

Q3:

In [8]:

```
list_mean = []
list_std = []
list_sr = []
list_skew = []
list_kurt = []
list_max = []
list_min = []
list_acf1 = []
list_acf12 = []
list_acf24 = []




list_test_factors = [test_MV_Qspread, test_BM_Qspread,
                     test_HL1M_Qspread,sp500['ret_sp500']]

for idx in range(len(list_test_factors)):
    list_mean.append(np.nanmean(list_test_factors[idx]))
    list_std.append(np.nanstd(list_test_factors[idx]))
    list_sr.append(np.sqrt(12)*list_mean[idx]/list_std[idx])
    list_skew.append(stats.skew(list_test_factors[idx]))
    list_kurt.append(stats.kurtosis(list_test_factors[idx]))
    list_max.append(np.nanmax(list_test_factors[idx]))
    list_min.append(np.nanmin(list_test_factors[idx]))
    list_acf1.append(sm.tsa.acf(list_test_factors[idx], nlags=24, fft=False)[1])
    list_acf12.append(sm.tsa.acf(list_test_factors[idx], nlags=24, fft=False)[12])
    list_acf24.append(sm.tsa.acf(list_test_factors[idx], nlags=24, fft=False)[24])


df_table = pd.DataFrame(list(zip(list_mean, list_std, list_sr, list_skew, list_kurt,
                                 list_max, list_min,  list_acf1, list_acf12, list_acf24)
                        columns =['Mean','STD','Sharpe Ratio','Skewness','Kurtosis',
                                  'Max','Min','ACF1','ACF12','ACF24'])
df_table.index = list_factors
df_table.T
```

Out[8]:

	MV	BM	HL1M	S&P500
Mean	-0.351353	0.380835	0.792846	0.007414
STD	3.989772	4.006618	3.607656	0.044229
Sharpe Ratio	-0.305060	0.329268	0.761297	0.580661
Skewness	-1.080343	0.309535	1.286733	-0.727039
Kurtosis	9.677470	4.671547	5.748761	2.097639
Max	19.610220	23.844838	20.645740	0.131767

Min	-30.725582	-18.943367	-9.717614	-0.217630
ACF1	0.172911	0.164694	-0.129980	0.023915
ACF12	0.140895	0.014665	-0.010097	0.034097
ACF24	0.064719	-0.022511	0.070606	0.050344

In [9]:
```
df_test_factor = pd.DataFrame(list(zip(test_MV_Qspread, test_BM_Qspread,
                        test_HL1M_Qspread, sp500['ret_sp500'])),columns = list_factors)

pd.DataFrame(np.corrcoef(df_test_factor.T), columns = list_factors,
             index = list_factors)
```

Out[9]:

	MV	BM	HL1M	S&P500
MV	1.000000	-0.731410	-0.469961	-0.328232
BM	-0.731410	1.000000	0.410794	0.187299
HL1M	-0.469961	0.410794	1.000000	0.340829
S&P500	-0.328232	0.187299	0.340829	1.000000

Q4:

In [10]:
```
list_mean = []
list_tstats = []
list_pvalue = []

for idx in range(len(list_test_factors)):
    list_mean.append(np.nanmean(list_test_factors[idx]))
    list_tstats.append(stats.ttest_1samp(list_test_factors[idx], 0)[0])
    list_pvalue.append(stats.ttest_1samp(list_test_factors[idx], 0)[1])

df_table_test = pd.DataFrame(list(zip(list_mean, list_tstats, list_pvalue)),
                        columns =['Mean', 'T Stats', 'p-Value'])
df_table_test.index = list_factors
df_table_test
```

Out[10]:

	Mean	T Stats	p-Value
MV	-0.351353	-1.828242	0.068204
BM	0.380835	1.973318	0.049098
HL1M	0.792846	4.562493	0.000007
S&P500	0.007414	3.479930	0.000553

In [ ]: